

LAMMPS pip-wheel build — complete reproduction reference

Part A: Docker / manylinux_2_28 wheel creation

Part B: Native apt build (Ubuntu / Pop!_OS)

Part C: Target machine runtime requirements

Part D: CUDA (GPU package) rebuild, LAMMPS 2026.9.30

Contents

I	Part A — Portable manylinux_2_28 wheel (Docker)	4
1	Pull the image and start the container	4
2	What is already in the image (no dnf action needed)	5
3	dnf packages to install inside the container	6
3.1	Core build dependencies (defensive install, most already in image)	6
3.2	EPEL repository (needed for voro++ and EPEL-only packages)	6
3.3	ffmpeg (RPM Fusion — not in base repos or EPEL on AlmaLinux 8)	7
3.4	Cython (for uv-managed Python, required by ML-IAP mliappy path)	7
4	Two Python interpreters — why both are needed	7
5	cmake configure — every error hit, in order, with fixes	8
5.1	Error 1: Python not found	8
5.2	Error 2 (transient): cmake not editing Python_EXECUTABLE	9
5.3	Non-blocking warnings (left as-is)	9
5.4	Error 3: dependency-not-found for downloadable packages	9
5.5	Error 4: stale cache — ADIOS2	10
5.6	FFMPEG: not in default repos, needs RPM Fusion	11
6	Final cmake configure command	11
7	Build	13
8	Build the wheel	13
9	auditwheel: inspect, then repair	14
9.1	Install auditwheel and inspect first	14
9.2	First repair attempt — and the two runtime failures it caused	14
9.3	Fix: exclude MPI and curl/ldap from vendoring	15

9.4	What remains bundled	16
10	Verification on target machine (Pop!_OS)	16
10.1	Installed-packages check	16
10.2	Multi-rank MPI test	17
10.3	NetCDF dump (exercises HDF5/krb5/gssapi chain)	17
10.4	Embedded Python package (PKG_PYTHON)	17
10.5	KIM API presence	18
11	Complete command sequence, start to finish	18
II	Part B — Native apt build (Ubuntu / Pop!_OS)	21
12	Base build tooling	21
13	CMake + cmake (Kitware repo)	21
14	Compilers and MPI (OpenMPI)	22
15	Python and wheel tooling	22
16	All libraries (MLIAP, EXTRA-PAIR, PYTHON, COLVARS, FFT, dump stack)	22
17	cmake configure for native build	23
18	Post-build sanity checks	24
19	Build the wheel (native)	25
20	Runtime smoke test (native build)	25
21	Portability caveats for the native-built wheel	26
III	Part C — Target machine runtime requirements	27
22	Debian / Ubuntu / Pop!_OS target	27
23	RHEL / AlmaLinux / Rocky Linux target	28
24	Install the wheel	28
25	KIM model data (either target)	28
26	Quick functional verification on target	29
27	Recommended: pin this wheel via a uv project	29
IV	Part D — CUDA (GPU package) rebuild, LAMMPS 2026.9.30	32

28 What was and was not tested	32
29 Differences from Part A	33
30 Source checkout	33
31 Container	34
32 Packages and tools	34
32.1 dnf	34
32.2 cmake and ccmake (Kitware binary)	35
32.3 CUDA toolkit	35
32.4 uv, Python and shell environment	36
33 Configure	37
33.1 Errors hit, in order	38
33.2 Record: xtb for QMMM-XTB was attempted and dropped	38
34 Build	38
35 Wheel, auditwheel repair and retag	39
35.1 First repair: Part A exclude list plus libcuda (fails at runtime)	39
35.2 Fix: exclude the whole MPI family	40
35.3 Retag: drop the Python-version lock	40
36 Verification	41
37 Target machine requirements for the CUDA wheel	42
38 Complete command sequence (2026.9.30, CUDA)	42
A Unrelated: A100 CUDA export fix	44

Part I

Part A — Portable manylinux_2_28 wheel (Docker)

Scope

Full, step-by-step reproduction record sufficient to recreate the portable `lammps` wheel from a clean Docker pull. Covers: image selection, *every* `dnf` package installed inside the container (and why), Python interpreter setup, `cmake` configure with every error encountered in order and its fix, build, wheel packaging via `python/install.py`, `auditwheel` repair (including the two runtime failures the first repair attempt caused and how they were resolved), and functional verification on a separate Pop!_OS target.

Source image: `quay.io/pypa/manylinux_2_28_x86_64:latest` (stock, unmodified, pulled 2026-08-08). Nothing was pre-baked; all packages were added via `dnf` inside a running container or fetched by `cmake`'s own `DOWNLOAD_*` mechanism.

LAMMPS version built and verified: 2026.2.11.

Update 2026-10-08: this part stays the record of the CPU-only 2026.2.11 build and is changed only by marked “Update” notes. The CUDA rebuild at 2026.9.30 is Part D; Section 29 lists everywhere it differs from this part.

1 Pull the image and start the container

Run from the directory that already contains the LAMMPS source tree (`cmake/`, `src/`, `python/`, etc.).

```
docker pull quay.io/pypa/manylinux_2_28_x86_64:latest

docker run -it --rm \
    -v "$PWD":/work \
    -w /work \
    quay.io/pypa/manylinux_2_28_x86_64:latest
```

`/work` is bind-mounted from the host. Anything written *outside* `/work` (e.g. `/root/...`) is **lost on container restart** — this matters for the uv-managed Python installed in Section 4.

Update 2026-10-08: the build container from this part was later lost (removed by mistake), so the next build started from scratch. Part D uses a named container without `-rm`, plus `docker commit` checkpoints (Section 31).

2 What is already in the image (no dnf action needed)

The following were confirmed present in the image pull used for this build — verified by `cmake`’s own “Found” lines during configure, not assumed:

- **Compiler:** `gcc-toolset-14` (`gcc/g++/gfortran 14.2.1`) at `/opt/rh/gcc-toolset-14/root/usr/bin/`
- **cmake:** pip-installed `cmake 4.4` (on `PATH`, supersedes the system’s `cmake 3.26.5`); system `cmake 3.26.5` *also* installed at `/usr/bin/cmake`
- **ccmake:** NOT in the pip-installed `cmake` wheel (no curses); **IS** present at `/usr/bin/ccmake` via the system `cmake` RPM. *Verify with:* `which -a cmake ccmake`
- **git** (required by `cmake DOWNLOAD_*` for source fetches)
- **MPI:** OpenMPI 3.1 headers and libs, `mpicxx` wrapper functional, found at `/usr/lib64/openmpi/`
- **OpenMP** 4.5
- **ZLIB** 1.2.11
- **FFTW3** 3.3.5 (threaded, found via `pkg-config`)
- **JPEG** (`libjpeg-turbo 62`), **PNG** (`libpng 1.6.34`)
- **CURL** 7.61.1 (`libcurl-devel` headers present)
- **NetCDF**, **LAPACK**/**OpenBLAS**, **GSL**, **HDF5**, **SWIG**, **BLAS**, **PkgConfig**, **Threads**
- **manylinux-blessed static Python 3.12** at `/opt/python/cp312-cp312/bin/python`

These are image-state dependent. For a reproducible future build, install the `dnf` packages in Section 3 regardless — they are no-ops if already satisfied, and will fail loudly if not.

Update 2026-10-08 — image drift. A re-pull of the same tag showed differences from the list above:

- `/usr/bin/cmake` (system 3.26.5) was absent; `which -a cmake ccmake` found only `/usr/local/bin/cmake` (pip, 4.4.3) and a `/usr/local/bin/ccmake` stub that raises `FileNotFoundError` (the pip wheel ships no `ccmake`).

- GSL was not found by PLUMED’s configure step (Could NOT find GSL); `gsl-devel` had to be installed.
- A second `uv` (0.12.22, `pipx` venv under `/opt/_internal/pipx/venvs/uv`) was already present and shadowed a fresh install.
- `cmake`’s “MPI found version 3.1” is the MPI *standard* version. The `dnf` package on this date was `openmpi-4.1.1-5.el8`, so the “OpenMPI 3.1” wording above is wrong for that image.

3 dnf packages to install inside the container

Install these in order. The groupings explain *why* each is needed.

3.1 Core build dependencies (defensive install, most already in image)

```
dnf install -y \
  openmpi openmpi-devel \
  fftw-devel \
  libjpeg-turbo-devel \
  libpng-devel \
  zlib-devel \
  libcurl-devel \
  netcdf-devel \
  openblas-devel \
  lapack-devel \
  ncurses-devel \
  git
```

Notes:

- `openmpi-devel` — headers and `mpicxx` wrapper; MPI was confirmed present in the image, but installing explicitly guarantees it across image updates.
- `ncurses-devel` — defensive for `ccmake`; not actually needed if `/usr/bin/ccmake` is already present.
- The rest map to libraries confirmed linked into `liblammps.so` by `auditwheel show`.

After installing `openmpi-devel`, confirm the `mpicxx` wrapper is on PATH:

```
which mpicxx || export PATH=/usr/lib64/openmpi/bin:$PATH
```

3.2 EPEL repository (needed for `voro++` and EPEL-only packages)

```
dnf install -y epel-release
dnf update -y
```

3.3 ffmpeg (RPM Fusion — not in base repos or EPEL on AlmaLinux 8)

```
dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm

dnf install -y ffmpeg
```

ffmpeg is never compiled against or linked into `liblammmps.so` — it is invoked by LAMMPS as an external subprocess at runtime for `dump movie`. It has no effect on wheel portability (`auditwheel` never touches it). Not installing it here means `cmake` silently leaves the `dump movie` feature off; installing it enables the feature.

Two-step fix required: after `dnf install -y ffmpeg`, `cmake` detects the binary but leaves `WITH_FFMPEG` off by default. Pass `-D WITH_FFMPEG=yes` explicitly in the configure command (Section 6).

3.4 Cython (for uv-managed Python, required by ML-IAP `mliappy` path)

This cannot be done until the uv Python is set up (Section 4). See that section for the correct invocation.

4 Two Python interpreters — why both are needed

This is the single most important structural fact about the manylinux wheel build. Two separate Python operations need Python for different reasons:

- **\$PYBIN** = `/opt/python/cp312-cp312/bin` — the manylinux-blessed CPython. Statically linked (`Py_ENABLE_SHARED=0`), **no** `libpython3.12.so` anywhere under its prefix (confirmed via `find /opt/python/cp312-cp312 -iname "libpython*" returning nothing`). Correct for stamping the wheel's ABI tag (`cp312-cp312-manylinux_2_28_x86_64`). **Cannot** satisfy `PKG_PYTHON=yes` which needs `Development.Embed` — an actual shared `libpython` to link against.
- **\$UVPY** — a uv-managed CPython (`python-build-standalone`). Ships `libpython3.12.so.1.0` (shared, `Py_ENABLE_SHARED=1`). Used *only* to satisfy the `cmake` configure/compile step for `PKG_PYTHON`.

Two-pass split: compile `liblammmps.so` using `$UVPY` (needs the shared lib), then build the actual wheel using `$PYBIN` (needs the manylinux-correct ABI tag). Once `liblammmps.so` is on disk, it carries no memory of which interpreter built it; the wheel-build step can use a different one.

Caveat: uv's installed Pythons live under `/root/.local/share/uv/...`, not under `/work`. This directory is **lost on container restart** — happened once during this build. `$UVPY` must be reinstalled and re-exported before continuing after any restart.

Set up both interpreters

```
Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv --version

# Install embedding-capable Python 3.12 via uv
uv python install 3.12
UVPY=$(uv python find 3.12)
echo $UVPY

# Verify it has the shared lib (must print 1)
$UVPY -c "import sysconfig; print(sysconfig.get_config_var('Py_ENABLE_SHARED'))"
find $(dirname $(dirname $UVPY)) -iname "libpython3.12*"

# Set the manylinux python (for wheel stamping and auditwheel)
export PYBIN=/opt/python/cp312-cp312/bin
$PYBIN/python --version
```

Install Cython under uv Python (required for ML-IAP mliappy)

PKG_ML-IAP's Python model path (mliappy) needs a working cythonize. Plain `$UVPY -m pip install cython` was tried first and rejected with `externally-managed-environment` (PEP 668, since uv manages this interpreter). The correct invocation is:

```
uv pip install --python $UVPY cython

# Confirm cythonize is discoverable
find $(dirname $(dirname $UVPY)) -name "cythonize"
```

Update 2026-10-08: with uv 0.12.23 this command fails: uv now marks its own managed Pythons as externally managed and refuses the install. Cython was *not* installed for the 2026.9.30 build and nothing broke, because the configure command sets `MLIAP_ENABLE_PYTHON=no` (mliappy off). Adding `-break-system-packages` to the command was suggested but never run.

5 cmake configure — every error hit, in order, with fixes

5.1 Error 1: Python not found

```
CMake Error: Could NOT find Python
(missing: Python_INCLUDE_DIRS Python_LIBRARIES
Interpreter Development Development.Module Development.Embed)
```

`Python_EXECUTABLE` was not specified on the first attempt, so `FindPython` searched default paths

and found nothing usable.

Fix: always pass `-D Python_EXECUTABLE=$UVPY` explicitly on every `cmake` invocation. See Section 4 for why `$UVPY` and not `$PYBIN`.

5.2 Error 2 (transient): `ccmake` not editing `Python_EXECUTABLE`

Attempting to change `PYTHON_EXECUTABLE` (old, deprecated `FindPythonInterp` name) inside `ccmake` silently failed. Modern CMake (3.12+ style `FindPython`) uses `Python_EXECUTABLE` — capital P.

Fix: do not fight through `ccmake`; wipe `build/` and pass `-D Python_EXECUTABLE=...` on the initial `cmake` invocation. Changing the interpreter after first configure does not reliably re-derive the cached `Python_LIBRARY/Python_INCLUDE_DIR`, risking a mismatched pairing that compiles but breaks at import.

5.3 Non-blocking warnings (left as-is)

These appeared during configure and did *not* stop the build — they only disable specific optional features:

- Could NOT find PNetCDF — parallel NetCDF variant, optional; core NetCDF still found and used.
- Could NOT find TBB_MALLOC — only affects Intel-package performance tuning (`dpd/intel`), not correctness.
- HDF5 C compiler wrapper unable to compile minimal program — warning only; HDF5 was still found and linked.

5.4 Error 3: dependency-not-found for downloadable packages

Each failed with a `find_package` error when the system package was absent. LAMMPS's `cmake` supports fetching and building each from source in-tree via a `DOWNLOAD_*` flag. Decision made mid-session: use `DOWNLOAD_*=yes` for *all* of these for reproducibility — every dependency gets the same low-glibc-baseline compile as the rest of the build, which is more portable than whatever a distro repo would have shipped.

```
Could NOT find VORO
n2p2 not found ... set DOWNLOAD_N2P2=ON
MDI download requested (no system package exists for MDI)
Could NOT find KIM-API
PLUMED / QUIP not found
Eigen3 not found
```

Fix: pass all of the following in the `cmake` invocation:

```
-D DOWNLOAD_VORO=yes
-D DOWNLOAD_N2P2=yes
-D DOWNLOAD_MDI=yes
-D DOWNLOAD_KIM=yes
```

```
-D DOWNLOAD_PLUMED=yes
-D DOWNLOAD_QUIP=yes
-D DOWNLOAD_EIGEN3=yes
```

Each `DOWNLOAD_X=yes` only takes effect if the corresponding `PKG_X=yes` (or the package-specific enable flag, e.g. `PKG_ML-HDNNP` for `n2p2`, `PKG_ML-QUIP` for `QUIP`) is also set. A `DOWNLOAD` flag alone with the package disabled does nothing.

Note: `vorop++/vorop++-devel` and `eigen3-devel` are available as real `dnf/apt` packages (via EPEL and default repos, respectively) and can be used as a system install if preferred. The `DOWNLOAD_*=yes` route was chosen here for a uniform low-glibc baseline.

```
System-package alternative (not used in this build, recorded for reference):
dnf install -y epel-release
dnf install -y vorop++ vorop++-devel
# then use: -D PKG_VORONOI=yes -D DOWNLOAD_VORO=no

# Eigen3 system package alternative:
dnf install -y eigen3-devel
# then omit DOWNLOAD_EIGEN3=yes
```

Update 2026-10-08: the 2026.9.30 build used these system packages (`vorop++-0.4.6`, `eigen3-devel-3.3.4`); the `DOWNLOAD_VORO` and `DOWNLOAD_EIGEN3` flags were dropped. `cmake` printed `Found VORO: /usr/lib64/libvorop++.so` and gave no Eigen version complaint. Whether `auditwheel` vendored `libvorop++` into that wheel was not checked.

5.5 Error 4: stale cache — ADIOS2

```
CMake Error: By not providing "FindADIOS2.cmake" ...
Could not find a package configuration file provided by "ADIOS2"
```

`PKG_ADIOS` was ON in `CMakeCache.txt` despite not being in the active flag list — leftover from an earlier attempt, since `build/` was not wiped before that particular reconfigure. No `DOWNLOAD_ADIOS2` flag exists in LAMMPS's `cmake` (unlike the packages above) — ADIOS2 must be a real system install if wanted.

Fix used: disabled it, since it was not actually needed:

```
cmake -B build -D PKG_ADIOS=no
```

To avoid this entirely: always wipe `build/` before a fresh configure (see Section 6).

5.6 FFMPEG: not in default repos, needs RPM Fusion

```
Could NOT find FFMPEG (missing: FFMPEG_EXECUTABLE)
```

```
Plain dnf fails:
dnf install -y ffmpeg
# => No match for argument: ffmpeg

# Add RPM Fusion, then install:
dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm
dnf install -y ffmpeg
```

After installing ffmpeg and reconfiguring, the binary is detected but the cmake feature variable remains off:

```
FFMPEG_EXECUTABLE:FILEPATH=/usr/bin/ffmpeg
WITH_FFMPEG:BOOL=OFF
```

Finding the binary and enabling the feature are two separate cmake variables. Pass `-D WITH_FFMPEG=yes` explicitly in the configure command.

6 Final cmake configure command

```
Start from /work (the LAMMPS source root)
cd /work
rm -rf build && mkdir build && cd build

cmake -B . -S ../cmake \
    -D BUILD_SHARED_LIBS=yes \
    -D CMAKE_BUILD_TYPE=Release \
    -D PKG_MLIAP=yes \
    -D PKG_EXTRA-PAIR=yes \
    -D PKG_KIM=yes \
    -D PKG_MDI=yes \
    -D PKG_PYTHON=yes \
    -D PKG_COLVARS=yes \
    -D PKG_NETCDF=yes \
    -D PKG_VORONOI=yes \
    -D PKG_ML-HDNNP=yes \
    -D PKG_PLUMED=yes \
    -D PKG_ML-QUIP=yes \
    -D PKG_ADIOS=no \
    -D MLIAP_ENABLE_PYTHON=no \
    -D BUILD_MPI=yes \
```

```

-D DOWNLOAD_KIM=yes \
-D DOWNLOAD_MDI=yes \
-D DOWNLOAD_N2P2=yes \
-D DOWNLOAD_VORO=yes \
-D DOWNLOAD_PLUMED=yes \
-D DOWNLOAD_QUIP=yes \
-D DOWNLOAD_EIGEN3=yes \
-D WITH_FFMPEG=yes \
-D Python_EXECUTABLE=$UVPY

```

Key flag notes:

- `MLIAP_ENABLE_PYTHON=no` is deliberate — this project needs `PKG_MLIAP` (C++/ML potentials) and `PKG_PYTHON` (importable from `lammps import lammps` module), but *not* Python-callable ML potentials at simulation runtime, which is a separate, narrower feature than either.
- `Python_EXECUTABLE=$UVPY` (not `$PYBIN`) — uv Python has `libpython3.12.so.1.0`; the manylinux python does not. `PKG_PYTHON=yes` requires it.
- `PKG_ADIOS=no` — ADIOS2 has no cmake `DOWNLOAD` flag and was not needed; this explicitly disables it even if it was cached from an earlier attempt.

Expected tail output:

```

Configuring done (X.Xs)
Generating done (X.Xs)
Build files have been written to: /work/build

```

With no `Configuring incomplete, errors occurred!` line. The full enabled-packages summary (including KIM, MDI, PYTHON, COLVARS, NETCDF, VORONOI, ML-HDNNP, PLUMED, ML-QUIP, ML-IAP, ML-PACE, etc.) and compiler/MPI/FFT settings block should print above that.

The actual enabled packages list from this build was: AMOEBA; APIP; ASPHERE; BOCS; BODY; BPM; BROWNIAN; CG-DNA; CG-SPICA; CLASS2; COLLOID; COLVARS; COMPRESS; CORESHELL; DIELECTRIC; DIFFRACTION; DIPOLE; DPD-BASIC; DPD-MESO; DPD-REACT; DPD-SMOOTH; DRUDE; EFF; ELECTRODE; EXTRA-COMMAND; EXTRA-COMPUTE; EXTRA-DUMP; EXTRA-FIX; EXTRA-MOLECULE; EXTRA-PAIR; FEP; GRANULAR; GRAPHICS; H5MD; INTEL; INTERLAYER; KIM; KSPACE; LATBOLTZ; LEPTON; MACHDYN; MANIFOLD; MANYBODY; MC; MDI; MEAM; MESONT; MGPT; MISC; ML-HDNNP; ML-IAP; ML-PACE; ML-POD; ML-QUIP; ML-RANN; ML-SNAP; ML-UF3; MOFFF; MOLECULE; MOLFILE; NETCDF; OPENMP; OPT; ORIENT; PERI; PHONON; PLUGIN; PLUMED; PTM; PYTHON; QEQ; QMMM; QTB; REACTION; REAXFF; REPLICAS; RHEO; RIGID; SHOCK; SMTBQ; SPH; SPIN; SRD; TALLY; UEF; VORONOI; YAFF

7 Build

```
make -j$(nproc)
```

This is a long build — KIM, N2P2, PLUMED, QUIP, PACE, MDI, Voro++, and Eigen3 all compile from source in-tree. Expect significant time.

Expected final lines:

```
[100%] Built target lammps
[100%] Built target lmp
```

A `pylammps/_pylammps.so` SWIG-generated target may also build (triggered by `PKG_PYTHON` + SWIG being present in the image). This is a separate, older binding mechanism, unrelated to the `python/lammps ctypes` package this document packages. Ignore it; it is not included in the wheel.

8 Build the wheel

`python/` already contains LAMMPS's own `install.py` and `setup.py` — no custom packaging script is needed. Switch to `$PYBIN` here (not `$UVPY`), so the wheel's ABI tag is `manylinux-correct`:

```
cd /work/python

$PYBIN/python install.py \
    -p lammps \
    -l ../build/liblammps.so \
    -v ../src/version.h \
    -n \
    -w .
```

Flag meanings:

- `-p lammps` — package directory name
- `-l ../build/liblammps.so` — path to the compiled shared lib
- `-v ../src/version.h` — LAMMPS version source file
- `-n` — build wheel only (do not install into the current env)
- `-w .` — output directory for the `.whl` file

`install.py` copies the `python/` tree to a scratch directory, copies `liblammps.so` into the `lammps/` package folder (so `setup.py`'s `package_data` picks it up), builds a venv from the invoking interpreter, and runs the wheel build inside that venv — calling it via `$PYBIN/python` is not cosmetic, it controls the wheel's ABI tag.

Output at this stage: `lammps-2026.2.11-cp312-cp312-linux_x86_64.whl` — correctly Python-tagged, but *not yet* portable (glibc symbol versions unconstrained, `linux_x86_64` generic tag).

9 auditwheel: inspect, then repair

9.1 Install auditwheel and inspect first

```
$PYBIN/pip install auditwheel patchelf

$PYBIN/python -m auditwheel show \
    lammps-2026.2.11-cp312-cp312-linux_x86_64.whl
```

Confirmed clean: no blacklisted symbols, resolves to `manylinux_2_27_x86_64`.

Note: `auditwheel show` does not list MPI libraries because it does not intend to vendor them by default policy. Confirm MPI is actually linked:

```
ldd ../build/liblammps.so | grep -i mpi
```

MPI was confirmed linked even though `auditwheel show` did not list it — the repair step below handles this.

9.2 First repair attempt — and the two runtime failures it caused

```
DO NOT USE THIS FORM --- documents the failure, not the fix
$PYBIN/python -m auditwheel repair \
    lammps-2026.2.11-cp312-cp312-linux_x86_64.whl \
    -w wheelhouse/
```

This repair, without any `-exclude` flags, vendored: MPI (`libmpi.so.40`, `libmpi_cxx.so.40`, and their transitive dependents), `libcurl.so.4`, `libldap-2.5.so.0`, `libkrb5`, and related libs (KIM's `kim query` pulls in `curl`, which transitively pulls `ldap/krb5`).

It also correctly stripped an explicit `libpython3.12.so.1.0` dependency from `liblammps.so` with the warning:

```
WARNING: Removing libpython3.12.so.1.0 dependency from lammps/liblammps.so.
Linking with libpython is forbidden for manylinux/musllinux wheels.
```

This is expected and correct.

Installed into a clean venv *on the container itself* — worked, `import lammps` returned ok.

Installed on the actual Pop!_OS target via `uv pip install` — **two separate failures:**

Failure 1 — MPI crash at import:

```
-----  
Sorry! You were supposed to get help about:  
  opal_init:startup:internal-failure  
But I couldn't open the help file:  
  /usr/lib64/openmpi/share/openmpi/help-opal-runtime.txt:  
  No such file or directory.  
-----  
*** An error occurred in MPI_Init  
*** on a NULL communicator
```

Cause: the bundled MPI libraries were the RHEL/AlmaLinux build, with internal data paths hardcoded to `/usr/lib64/openmpi/share/...` — a path layout that does not exist on Debian-family Pop!_OS. Pop!_OS already had its own working OpenMPI 4.1.2 installed, entirely unused because the bundled copy's RPATH took priority.

Failure 2 — segfault at interpreter shutdown (after a clean run):

```
Segmentation fault (11)  
...  
/usr/lib/x86_64-linux-gnu/libc.so.6(free+0x1e)  
/usr/lib/x86_64-linux-gnu/libldap-2.5.so.0(+0x1007b)
```

Cause: the bundled libldap and the system's own separately-loaded libldap-2.5.so.0 (pulled in via NSS at process exit) both get unloaded at teardown. A version/ABI mismatch triggers a double-free inside libc's `free()`. Same failure class as MPI — a bundled library colliding with a system-loaded library of the same effective identity at a point `auditwheel`'s static analysis cannot see (runtime `dlopen`/NSS-plugin behavior, not a direct `DT_NEEDED` entry).

9.3 Fix: exclude MPI and curl/ldap from vendoring

```
cd /work/python/wheelhouse  
  
$PYBIN/python -m auditwheel repair \  
  ../lammps-2026.2.11-cp312-cp312-linux_x86_64.whl \  
  -w . \  
  --exclude libmpi.so.40 \  
  --exclude libmpi_cxx.so.40 \  
  --exclude libldap-2.5.so.0 \  
  --exclude libcurl.so.4
```

Excluding `libmpi.so.40/libmpi_cxx.so.40` automatically dropped their transitive dependents (`libopen-rte.so.40`, `libopen-pal.so.40`, `libopen-orted-mpir.so`) without needing to list them separately. This produces a wheel that *requires* the target system to have its own compatible OpenMPI and curl/ldap installed — the correct contract for an MPI-linked wheel.

Update 2026-10-08 — this list was not enough for 2026.9.30. Excluding only `libmpi.so.40` and `libmpi_cxx.so.40` did *not* drop the rest of the MPI family in that build. `auditwheel` vendored the Fortran bindings (`libmpi_mpifh`, `libmpi_usempif08`, `libmpi_usempi_ignore_tkr`) together with `libopen-rte`, `libopen-pal`, `libopen-orted-mpir`, `libhwloc` and two `libevent` libraries, and `import lammps` on `Pop!_OS` aborted in `MPI_Init` with the same `/usr/lib64/openmpi/share/openmpi/` help-file error as Failure 1. The bindings show up in `ldd` of `liblammps.so`; whether as direct or transitive dependencies was not checked. The full exclude list is in Section 35; use it for any rebuild, CPU or GPU.

Result:

```
Fixed-up wheel written to:
lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```

Note: `auditwheel` found the wheel was eligible for the higher-priority `manylinux_2_27` tag and used both tags.

9.4 What remains bundled

After the fix, `lammps.libs/` on the target contained: `FFTW3(+omp)`, `OpenBLAS`, `GSL/GSLCBLAS`, `HDF5(+HL)`, `NetCDF`, `libkim-api`, `libjpeg`, `libpng16`, `libzstd`, `libgfortran`, `libquadmath`, `libgomp`, plus `libgssapi_krb5/libkrb5/libselinux/libtirpc/libcrypto` (transitive, possibly from `HDF5/NetCDF` auth features rather than `curl`, since `curl` itself was excluded). These were not excluded because the functional tests below exercised the codepaths that would trigger them (`netCDF` dump, `KIM`) and passed cleanly with exit code 0.

10 Verification on target machine (Pop!_OS)

Copy the wheel file out of the container (or from the bind-mounted `/work` directory) to the target machine, then:

```
uv pip install --force-reinstall \
    lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```

`-force-reinstall` is required when re-testing a rebuilt wheel with an unchanged version string, or `uv/pip` will skip the reinstall.

10.1 Installed-packages check

```
python -c "from lammps import lammps; l = lammps(); print(l.installed_packages)"
echo $?
```

Expected: full compiled-in package list printed, exit code 0. Confirmed output included `KIM`, `MDI`, `PYTHON`, `COLVARS`, `NETCDF`, `VORONOI`, `ML-HDNNP`, `PLUMED`, `ML-QUIP`, `ML-IAP`,

ML-PACE.

10.2 Multi-rank MPI test

```
mpirun -np 2 python -c \  
    "from lammps import lammps; l = lammps(); \  
    l.commands_string('units lj\natom_style atomic\n\  
lattice fcc 0.8442\nregion box block 0 4 0 4 0 4\n\  
create_box 1 box\ncreate_atoms 1 box\nmass 1 1.0\n\  
pair_style lj/cut 2.5\npair_coeff 1 1 1.0 1.0 2.5\nrun 10'); \  
    print('mpi ok')" -quiet  
echo "exit: $?"
```

Passed on both ranks with identical energy/pressure output to a single-rank run. Exit code 0.

10.3 NetCDF dump (exercises HDF5/krb5/gssapi chain)

```
python -c "  
from lammps import lammps  
l = lammps()  
l.commands_string(''  
units lj  
atom_style atomic  
lattice fcc 0.8442  
region box block 0 4 0 4 0 4  
create_box 1 box  
create_atoms 1 box  
mass 1 1.0  
pair_style lj/cut 2.5  
pair_coeff 1 1 1.0 1.0 2.5  
dump 1 all netcdf 5 test.nc id type x y z  
run 10  
''')  
print('netcdf ok')  
"  
echo "exit: $?"
```

Passed, exit code 0.

10.4 Embedded Python package (PKG_PYTHON)

```
python -c "  
from lammps import lammps  
l = lammps()  
l.commands_string(''  
python test_fn input 0 return v_result format f here \"%\"  
def test_fn():
```

```

        return 1.0
    \""\"
    '')
print('python-embed ok')
"
echo "exit: $?"

```

Passed, exit code 0.

10.5 KIM API presence

```

python -c "
from lammps import lammps
l = lammps()
print('KIM present:', 'KIM' in l.installed_packages)
"

```

Confirmed True. Only the API/library was verified here; no actual KIM model data was installed or tested — model data is a separate step (Section ??).

11 Complete command sequence, start to finish

```

-- 1. Start container (from LAMMPS source root on host) -----
docker run -it --rm -v "$PWD":/work -w /work \
    quay.io/pypa/manylinux_2_28_x86_64:latest

cd /work

# -- 2. Install dnf packages inside container -----
dnf install -y \
    openmpi openmpi-devel \
    fftw-devel \
    libjpeg-turbo-devel \
    libpng-devel \
    zlib-devel \
    libcurl-devel \
    netcdf-devel \
    openblas-devel \
    lapack-devel \
    ncurses-devel \
    git

# Confirm mpicc/mpicxx on PATH
which mpicxx || export PATH=/usr/lib64/openmpi/bin:$PATH

# EPEL for voro++-devel (optional; DOWNLOAD_VORO=yes is used instead here)
dnf install -y epel-release

```

```

# ffmpeg from RPM Fusion
dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm
dnf install -y ffmpeg

# -- 3. uv + embedding-capable Python -----
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv python install 3.12
UVPY=$(uv python find 3.12)
export PYBIN=/opt/python/cp312-cp312/bin

# Cython under uv Python (for ML-IAP mliappy)
uv pip install --python $UVPY cython

# -- 4. Configure -----
rm -rf build && mkdir build && cd build
cmake -B . -S ../cmake \
    -D BUILD_SHARED_LIBS=yes \
    -D CMAKE_BUILD_TYPE=Release \
    -D PKG_MLIAP=yes \
    -D PKG_EXTRA-PAIR=yes \
    -D PKG_KIM=yes \
    -D PKG_MDI=yes \
    -D PKG_PYTHON=yes \
    -D PKG_COLVARS=yes \
    -D PKG_NETCDF=yes \
    -D PKG_VORONOI=yes \
    -D PKG_ML-HDNNP=yes \
    -D PKG_PLUMED=yes \
    -D PKG_ML-QUIP=yes \
    -D PKG_ADIOS=no \
    -D MLIAP_ENABLE_PYTHON=no \
    -D BUILD_MPI=yes \
    -D DOWNLOAD_KIM=yes \
    -D DOWNLOAD_MDI=yes \
    -D DOWNLOAD_N2P2=yes \
    -D DOWNLOAD_VORO=yes \
    -D DOWNLOAD_PLUMED=yes \
    -D DOWNLOAD_QUIP=yes \
    -D DOWNLOAD_EIGEN3=yes \
    -D WITH_FFMPEG=yes \
    -D Python_EXECUTABLE=$UVPY

# -- 5. Build -----
make -j$(nproc)

# -- 6. Package the wheel -----
cd /work/python

$PYBIN/python install.py \
    -p lammmps \

```

```

-l ../build/liblammps.so \
-v ../src/version.h \
-n \
-w .

# -- 7. auditwheel repair -----
$PYBIN/pip install auditwheel patchelf

mkdir -p wheelhouse
$PYBIN/python -m auditwheel repair \
    lammps-*-cp312-cp312-linux_x86_64.whl \
    -w wheelhouse/ \
    --exclude libmpi.so.40 \
    --exclude libmpi_cxx.so.40 \
    --exclude libldap-2.5.so.0 \
    --exclude libcurl.so.4

ls wheelhouse/
# Expected: lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64
    .whl

# -- 8. Install on target (run on target machine, not in container) -----
# uv pip install lammps-*-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
# pip install lammps-*-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl

```

Part II

Part B — Native apt build (Ubuntu / Pop!_OS)

Scope

Package list for building LAMMPS natively on Ubuntu/Pop!_OS (no Docker, no manylinux container). This produces a *non-portable* wheel — it links against this machine's glibc and does not have a manylinux tag. Fine on this same OS; risky elsewhere (see Section 21).

This build was confirmed working on Pop!_OS with Python 3.12. Only LAMMPS itself is built from source. Everything it depends on comes from apt.

12 Base build tooling

```
sudo apt update
sudo apt install -y --no-install-recommends \
    ca-certificates gnupg wget curl git \
    software-properties-common apt-transport-https lsb-release \
    build-essential ninja-build pkg-config
```

13 CMake + ccmake (Kitware repo)

The distro-shipped cmake is typically too old. Use the Kitware repo:

```
wget -O - https://apt.kitware.com/keys/kitware-archive-latest.asc 2>/dev/null \
| gpg --dearmor - \
| sudo tee /usr/share/keyrings/kitware-archive-keyring.gpg >/dev/null

echo "deb [signed-by=/usr/share/keyrings/kitware-archive-keyring.gpg] \
https://apt.kitware.com/ubuntu/ $(lsb_release -cs) main" \
| sudo tee /etc/apt/sources.list.d/kitware.list

sudo apt update
sudo apt install -y cmake cmake-curses-gui

cmake --version && ccmake --version
```

cmake-curses-gui is the package that ships ccmake. The pip cmake wheel is built without curses and will not give you it.

14 Compilers and MPI (OpenMPI)

```
sudo apt install -y \  
    gcc g++ gfortran \  
    libopenmpi-dev openmpi-bin openmpi-common
```

If cmake reports Could NOT find MPI (missing: MPI_MPICXX_FOUND MPICXX) despite MPI being installed, the issue is almost always libopenmpi-dev/openmpi-bin present but the mpicxx wrapper compiler missing — the line above covers it.

Any target machine you later copy this wheel to needs an ABI-compatible OpenMPI runtime installed, or import lammps fails at load time on an unresolved libmpi.so symbol.

15 Python and wheel tooling

```
sudo apt install -y \  
    python3 python3-dev python3-pip python3-venv python3-full  
  
python3 -m pip install --break-system-packages \  
    --upgrade pip build wheel auditwheel patchelf
```

python3.12-dev note

python3-dev normally pulls in headers for whatever python3 defaults to. If cmake or the embed step can't find Python.h specifically for 3.12:

```
sudo apt install python3.12-dev
```

This is the package that ships Python.h and libpython3.12.so headers/import libs when the generic python3-dev isn't resolving to 3.12.

16 All libraries (MLIAP, EXTRA-PAIR, PYTHON, COLVARS, FFT, dump stack)

```
sudo apt install -y \  
    libfftw3-dev \  
    libeigen3-dev \  
    libcurl4-openssl-dev \  
    zlib1g-dev \  
    libjpeg-dev \  
    libpng-dev \  
    libzstd-dev
```

```
libnetcdf-dev \  
libpnetcdf-dev \  
libgsl-dev \  
libopenblas-dev \  
libhdf5-serial-dev \  
libkim-api-dev \  
cython3 \  
ffmpeg
```

Notes:

- `cython3` — ML-IAP's Python model path (`mliappy`) needs a working `cythonize`.
- `libnetcdf-dev` / `libpnetcdf-dev` — matches `LMP_HAS_NETCDF`; `pnetcdf` resolves the `Could NOT find PNetCDF` cmake error.
- `libgsl-dev`, `libopenblas-dev`, `libhdf5-serial-dev` — confirmed present in the working build's `ldd liblammps.so` output (`COLVARS/LEPTON` pulls `GSL`; `BLAS` and `HDF5` come in via the `netCDF`/other package chain).
- `libkim-api-dev` — system `KIM`. Use this with `-D DOWNLOAD_KIM=no` in cmake to install `KIM` as a normal system library rather than building it in-tree. This is what makes `libkim-api.so.2` resolve to `/usr/lib/x86_64-linux-gnu/libkim-api.so.2` instead of a build-tree path that disappears when the build directory is deleted.
- `ffmpeg` — runtime binary `LAMMPS` pipes movie dumps to; also needed at cmake-configure time for detection.

17 cmake configure for native build

```
rm -rf build && mkdir build && cd build

cmake -B . -S ../cmake \
  -D BUILD_SHARED_LIBS=yes \
  -D CMAKE_BUILD_TYPE=Release \
  -D PKG_MLIAP=yes \
  -D PKG_EXTRA-PAIR=yes \
  -D PKG_KIM=yes \
  -D PKG_MDI=yes \
  -D PKG_PYTHON=yes \
  -D PKG_COLVARS=yes \
  -D PKG_NETCDF=yes \
  -D PKG_VORONOI=yes \
  -D MLIAP_ENABLE_PYTHON=no \
  -D BUILD_MPI=yes \
  -D DOWNLOAD_KIM=no \
  -D DOWNLOAD_MDI=yes \
  -D Python_EXECUTABLE=$(which python3.12)

make -j$(nproc)
```

Key differences from the manylinux build:

- `DOWNLOAD_KIM=no` — use `libkim-api-dev` from apt; do not build an in-tree copy that would become unresolvable the moment `build/` is deleted.
- `Python_EXECUTABLE=$(which python3.12)` — use the system Python; no uv setup needed. The system Python has a shared libpython.
- `DOWNLOAD_VORO=yes` omitted — can use system `libvoro++-dev` (apt install `libvoro++-dev`), or add `-D PKG_VORONOI=yes -D DOWNLOAD_VORO=yes` for the in-tree build.

18 Post-build sanity checks

1. Confirm a shared lib was produced

```
grep BUILD_SHARED_LIBS build/CMakeCache.txt
ls -la build/liblammps*
```

Expected: `BUILD_SHARED_LIBS:BOOL=YES` (or `ON`) and a `liblammps.so` file present — not just `liblammps.a`. If it's `OFF` or only the `.a` exists, reconfigure with `-D BUILD_SHARED_LIBS=yes` and rebuild; nothing past this point works off a static-only build.

2. Confirm the binary runs

```
build/lmp -h
```

Expected: help/usage text with package list.

3. Check the shared-lib dependency chain

```
ldd build/liblammps.so
```

Expected: every entry resolves to a real path, no "not found" lines. With `libkim-api-dev` installed and `DOWNLOAD_KIM=no` set, `libkim-api.so.2` should resolve to `/usr/lib/x86_64-linux-gnu/libkim-api.so.2` — a normal system path. If it still points into your build tree, `cmake` picked up the in-tree copy anyway; reconfigure with `DOWNLOAD_KIM=no` and confirm `KIM-API_FOUND` in `cmake` output.

19 Build the wheel (native)

```
cd build
make install-python
```

Or, to get just the `.whl` without installing into the current env:

```
python3 ../python/install.py \
  -n \
  -p ../python/lammps \
  -l liblammps.so \
  -v ../src/version.h \
  -w .
```

20 Runtime smoke test (native build)

```
python3 -c "import lammps; l = lammps.lammps(); print('ok')"
```

Expected output (confirmed):

```
LAMMPS (11 Feb 2026)
  using 1 OpenMP thread(s) per MPI task
```

```
ok
Total wall time: 0:00:00
```

21 Portability caveats for the native-built wheel

This wheel is *not* `auditwheel`-repaired and dynamically links system libraries as-is:

- **glibc floor** — carries whatever glibc this Pop!_OS toolchain compiled against. An older-glibc target (e.g. older RHEL/CentOS) can fail to import.
- **OpenMPI** — target needs an ABI-compatible OpenMPI runtime.
- **KIM, now resolved** — with `libkim-api-dev` installed and `DOWNLOAD_KIM=no`, KIM resolves as a normal system library. The original landmine (unresolvable `libkim-api.so.2` path pointing into a deleted build tree) only applied to `DOWNLOAD_KIM=yes`.
- **Python-version lock** — this wheel is compiled against one specific Python minor version (3.12). It will not import under 3.11, 3.13, or any other. Every machine this wheel goes to needs a 3.12 interpreter.

For a portable wheel: use Part A (manylinux Docker build) instead.

Part III

Part C — Target machine runtime requirements

Scope

These packages are *not* bundled in the manylinux wheel and must be present on any target machine before `pip install / uv pip install` of the wheel, or `import lammps` will fail or crash at exit exactly as it did on Pop!_OS before the exclude fix.

Python: depends on the wheel. The `cp312`-tagged wheel (2026.2.11, and the first 2026.9.30 file) needs Python 3.12 specifically — it will not install under 3.11/3.13/anything else. The retagged `py3-none` 2026.9.30 wheel (Part D, Section 35.3) has `Requires-Python: >=3.6`; it was tested on 3.12 and 3.10.12, and the embedded LAMMPS `python` command is only considered safe under 3.12. Confirm before installing a `cp312` wheel:

```
python3 --version    # must print 3.12.x
```

22 Debian / Ubuntu / Pop!_OS target

```
sudo apt update
sudo apt install -y \
    libopenmpi3 openmpi-bin \
    libcurl4 \
    libldap-2.5-0 \
    ffmpeg
```

Notes:

- `libopenmpi3` — runtime MPI library, matches the v40-series ABI the wheel links against.
- `openmpi-bin` — provides `mpirun`, needed for multi-rank use.
- `libcurl4` / `libldap-2.5-0` — excluded from vendoring to prevent the shutdown-time seg-fault; must be present on the system. Both were already present by default on Pop!_OS and confirmed sufficient.
- `ffmpeg` — optional, only needed for `dump movie`.

This is the set that was **verified working** on the actual Pop!_OS target machine in this build.

For the CUDA wheel (Part D) the target also needs an NVIDIA driver; see Section 37.

23 RHEL / AlmaLinux / Rocky Linux target

```
sudo dnf install -y \  
    openmpi \  
    libcurl \  
    openldap  
  
# ffmpeg from RPM Fusion (not in base/EPEL on RHEL-family):  
sudo dnf install -y \  
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm  
sudo dnf install -y ffmpeg
```

Note: not verified end-to-end on an actual RHEL-family target in this session (only Pop!_OS was tested). Package names are correct for the family but treat this as unverified relative to the Debian section above, which was directly confirmed.

24 Install the wheel

```
Any of these are equivalent once runtime deps above are installed:  
uv pip install lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.  
    manylinux_2_28_x86_64.whl  
pip install lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.  
    manylinux_2_28_x86_64.whl
```

2026.9.30 CUDA wheel (Part D), retagged, no Python-version lock:

```
uv pip install lammps-2026.9.30-py3-none-manylinux_2_27_x86_64.manylinux_2_28_x86_64  
    .whl
```

25 KIM model data (either target)

The wheel ships the KIM API library only, not model data. If KIM potentials are used at runtime:

```
kim-api-collections-management install user <model-name>
```

26 Quick functional verification on target

```
Basic import and package list
python -c "from lammps import lammps; l = lammps(); print(l.installed_packages)"
echo "exit: $?"

# MPI
mpirun -np 2 python -c \
    "from lammps import lammps; l = lammps(); \
    l.commands_string('units lj\natom_style atomic\nlattice fcc 0.8442\n\
region box block 0 4 0 4 0 4\ncreate_box 1 box\ncreate_atoms 1 box\n\
mass 1 1.0\npair_style lj/cut 2.5\npair_coeff 1 1 1.0 1.0 2.5\nrun 10'); \
    print('mpi ok')" -quiet
echo "mpi exit: $?"
```

27 Recommended: pin this wheel via a uv project

The Python-version lock (3.12 only) has a practical consequence when running multiple MLIP backends (MACE, SevenNet, ORB, FAIRChem, PET-MAD) against this LAMMPS build. The clean way to manage this is a single `uv` project with the wheel referenced by local path in `[tool.uv.sources]`, `requires-python` pinned to 3.12, and each MLIP backend declared as a mutually-exclusive optional extra:

```
[project]
name = "mlip"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12,<3.13"
dependencies = [
    "ase>=3.27.0",
    "cmake>=4.2.3",
    "cryspy>=0.10.0",
    "cupy-cuda12x>=14.0.1",
    "dill>=0.4.1",
    "dpmdtools>=1.1.2",
    "elastic>=5.2.5.3",
    "fitz>=0.0.1.dev2",
    "frontend>=0.0.3",
    "gpu4pyrcf-cuda12x>=1.7.0",
    "imageio[ffmpeg]>=2.37.3",
    "lammps",
    "ligpargen>=2.1",
    "mdanalysis>=2.10.0",
    "mendelev>=1.1.0",
    "moltemplate>=2.22.5",
    "networkx>=3.6.1",
    "notebook>=7.5.5",
    "opencv-python>=4.13.0.92",
```

```

"openmm[cuda12]>=8.5.1",
"phono3py>=3.28.0",
"phonopy>=3.1.0",
"pymatgen>=2025.10.7",
"pymupdf>=1.27.2.2",
"pyqt6>=6.10.2",
"python-pptx>=1.0.2",
"rdkit>=2025.9.6",
"skala>=2026.1",
"spglib>=2.7.0",
"statsmodels>=0.14.6",
]
[project.optional-dependencies]
mace = [
    "mace-torch[cueq,cueq-cuda-12]",
    "cuequivariance>=0.11.0",
    "cuequivariance-torch>=0.11.0",
    "cuequivariance-ops-torch-cu12>=0.11.0",
]
sevensnet = ["sevensnet[cueq12]>=0.13.0"]
orb = ["orb-models>=0.5.5"]
fairchem = ["fairchem-core>=2.20.0"]
petmad = ["pet-mad"]
[tool.uv]
conflicts = [
    [
        { extra = "mace" },
        { extra = "sevensnet" },
        { extra = "orb" },
        { extra = "fairchem" },
        { extra = "petmad" },
    ],
]

[tool.uv.sources]
lammps = { url = "https://github.com/sampk1203/lammps-mlip-wheel/releases/download/
v2026.2.11/lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.
manylinux_2_28_x86_64.whl" }

```

Why this matters:

- `requires-python = ">=3.12,<3.13"` enforces the 3.12-only constraint at project-resolution level — uv refuses to solve against any other interpreter.
- `tool.uv.sources.lammps` points at the `.whl` directly by path instead of pulling from PyPI — lammps becomes a normal declared dependency, not a manual `pip install /path/to/wheel.whl` step to repeat in every new env.
- `tool.uv.conflicts` stops uv from trying to resolve two MLIP backends into the same lock simultaneously. Switch backends with `uv sync -extra mace / uv sync -extra orb` etc.

Update 2026-10-08: the pin above is for the 2026.2.11 CPU wheel. The py3-none 2026.9.30 wheel (Part D) has no Python-version lock, so LAMMPS no longer forces the `requires-python`

pin; this was not tested inside a uv project, and other dependencies there may still constrain the version. To switch to it, point the source at the new release asset:

```
lamps = { url = "https://github.com/sampk1203/lamps-mlip-wheel/releases/download/  
v2026.9.30/lamps-2026.9.30-py3-none-manylinux_2_27_x86_64.manylinux_2_28_x86_64  
.whl" }
```

Part IV

Part D — CUDA (GPU package) rebuild, LAMMPS 2026.9.30

Scope

Record of rebuilding the wheel with the LAMMPS GPU package (CUDA back end), done 2026-10-08 in a fresh container. It follows Part A and lists only what differs from it; Section 29 collects the places where Part A is wrong or outdated for this build. Parts A–C were not re-derived: Part A remains the record of the CPU-only 2026.2.11 build.

- **Source:** git tag `stable_30Sep2026`, commit `8de817dd79` (2026-09-30). Version string `2026.9.30`; the Python module reports `20260930`.
- **Toolchain:** `manylinux_2_28` image (AlmaLinux 8.10), `gcc-toolset-14 14.2.1`, CUDA toolkit 13.4 (`nvcc 13.4.92`), `cmake 4.4.3`.
- **GPU options:** `GPU_API=cuda`, `GPU_PREC=mixed`, `GPU_ARCH=sm_86`. `FFT_SINGLE` is *not* set, so FFTs stay double precision (cmake summary: “Using double precision FFTs”).
- **Output:** one repaired wheel, about 68 MB, in two tags: the original `cp312` file and a retagged `py3-none` copy (Section 35.3).

28 What was and was not tested

Tested — RTX 3050 Laptop GPU (compute capability 8.6, 4 GB), driver 580.173.02 (`nvidia-smi` reports CUDA 13.0), `Pop!_OS` host:

- `import lammps` works on Python 3.12 (`cp312` wheel) and on Python 3.10.12 (retagged `py3-none` wheel). Version `20260930`; GPU device found. `installed_packages` lists 89 packages including GPU and `VORONOI`; `QMMM` is present, `QMMM-XTB` and `FENIX` are not.
- Short LJ run with `-sf gpu -pk gpu 1` on Python 3.12 (4000 atoms, 100 steps): GPU package initialised on device 0 (RTX 3050 Laptop GPU, 16 CUs, mixed precision), kernels compiled and ran, loop time 0.0206791 s, no errors. This shows the kernels launch. It is not a CPU-versus-GPU accuracy comparison.

Not tested:

- RTX A1000 (cc 8.6, driver 595.91.07), A100 80 GB (cc 8.0, driver 595.91.07), T4 (cc 7.5, not reachable). The wheel was built with `GPU_ARCH=sm_86`, so native code exists for `sm_86` only. LAMMPS documentation says the CUDA build also tries to include the other architectures the toolkit supports, with a driver JIT delay at start-up. No `cuobjdump` check of the built library was done.
- PPPM and bonded styles under `-sf gpu`, the `pair/only` package keyword, multi-rank MPI together with the GPU, and any real input (e.g. `in_equilibrate.lammps`) against a CPU run. Mixed precision was not validated against double precision for stress/Green–Kubo output.

- Python 3.13, 3.14, 3.15, and the embedded LAMMPS `python` command on any version other than 3.12.
- Whether `libldap/libkrb5` were vendored into this wheel (the first repair log showed an `Excluding` line for `libcurl.so.4` but none for `libldap-2.5.so.0`).

29 Differences from Part A

- **Container:** named, no `-rm`, checkpointed with `docker commit` (Section 31).
- `dnf update -y` (EPEL step, Section 3) was skipped: it upgrades the whole image and makes rebuilds less reproducible. The EPEL and RPM Fusion installs are unchanged.
- **cmake/ccmake:** the image had no system `cmake` and the pip wheel has no `ccmake`; the Kitware binary was installed (Section 32).
- **GSL** is not in the image (Part A, Section 2, says it is); PLUMED’s configure step failed until `gsl-devel` was installed.
- **voro++ and Eigen** come from `dnf`, not `DOWNLOAD_*`.
- **Cython** was skipped (`uv` refuses the Part A command; not needed with `MLIAP_ENABLE_PYTHON=no`). See the update note in Section 3.4.
- **Configure:** `-C ../cmake/presets/most.cmake` was used. `PKG_MLIAP=yes` was dropped (the package name is `ML-IAP`, which the preset enables). Packages needing external libraries had to be switched off (Section 33). Part A’s configure command has no preset; whether that command alone reproduces Part A’s 87-package list was not checked.
- **auditwheel:** the Part A exclude list was insufficient; see Section 35.
- **Wheel tag:** retagged `py3-none` (Section 35.3).
- **Enabled packages:** 89 versus Part A’s 87. Added GPU and GRANSURF; `ML-RANN` is now `ML-RUNNER`.
- **install.py side effect:** it deleted the older `lammps-2026.2.11-cp312-cp312-linux_x86_64.whl` from `/work/python` (“Purging existing wheels”). The repaired 2026.2.11 wheel in `wheelhouse/` was not affected.

30 Source checkout

The working tree was on branch `release` with no modified tracked files, and the newest stable tag had the same date as `HEAD`. Custom untracked files (`src/pair_gnnp*`, `src/gnnp_driver.py`, `python/lammps/gnnp_driver.py`, `src/fairchem-omat24/`, `src/mace-osaka24/`, `python/wheelhouse/`) survive a tag switch; git only aborts if an incoming file collides with an untracked path. Run on the host, in the source root:

```
git fetch --tags -q
git tag --list 'stable_*' --sort=-creatordate | head -5
git rev-list --count stable_30Sep2026..HEAD
git checkout stable_30Sep2026
```

```
git log -1 --format='%h %ad' --date=short
```

The count printed 0, and the checkout landed on 8de817dd79 (detached **HEAD**, expected for a tag). Do not **git pull** while on the tag; **git switch release** returns to the branch. Always **rm -rf build** after a version change (stale cache, as in Part A Section 5).

gnnp_driver.py: install `python/lammps/gnnp_driver.py` (12104 bytes, 2026-08-07, md5 e6c50bcc...) into the wheel. A newer copy sits in `wheelhouse/` (14914 bytes, 2026-08-12, md5 d9dfb56d...). The driver is run from the directory containing the LAMMPS input file, so the copy inside the wheel is not used and was left as is.

31 Container

```
docker pull quay.io/pypa/manylinux_2_28_x86_64:latest

docker run -it --name lmpbuild -v "$PWD":/work -w /work \
    quay.io/pypa/manylinux_2_28_x86_64:latest

docker start -ai lmpbuild
```

The last command re-enters the container after an exit. No `-gpus` flag: building needs no GPU, and the container has no NVIDIA driver.

Checkpoints. From a second host terminal: `docker commit lmpbuild lmpbuild:<tag>`. A commit saves the container filesystem (dnf packages, CUDA, uv Python, `~/.bashrc`) but *not* `/work`: that is a bind mount, so sources, `build/` and wheels live on the host. Final image: `lmpbuild:wheel-done`, 5.04 GB. Files created in `/work` by the container are owned by root on the host.

32 Packages and tools

32.1 dnf

```
dnf install -y openmpi openmpi-devel fftw-devel libjpeg-turbo-devel \
    libpng-devel zlib-devel libcurl-devel netcdf-devel openblas-devel \
    lapack-devel ncurses-devel git

export PATH=/usr/lib64/openmpi/bin:$PATH
mpicxx --version | head -1

dnf install -y epel-release
dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm
dnf install -y ffmpeg

dnf install -y voro++ voro++-devel eigen3-devel gsl-devel libzstd-devel
```

`mpicxx` was not on `PATH` in the fresh container until the export above; it then wrapped `g++ 14.2.1`. `libzstd-devel` adds the `LAMMPS_ZSTD` define (zstd-compressed dumps); without it configure only warns.

32.2 cmake and ccmake (Kitware binary)

The pip `cmake 4.4.3` has no `ccmake`, and installing the system `cmake` gives 3.26.5 (removed again). The Kitware tarball includes both tools and matches the pip version, so there is one `cmake` version for configure and `ccmake`. Its URL pattern was written from memory and worked.

```
curl -fL -o /tmp/cmake.tgz \
  https://github.com/Kitware/CMake/releases/download/v4.4.3/cmake-4.4.3-linux-
  x86_64.tar.gz
mkdir -p /opt/cmake-4.4.3
tar -xzf /tmp/cmake.tgz -C /opt/cmake-4.4.3 --strip-components=1
for t in cmake ccmake ctest cpack; do
  ln -sf /opt/cmake-4.4.3/bin/$t /usr/local/bin/$t
done
hash -r
cmake --version | head -1
ccmake --version | head -1
```

32.3 CUDA toolkit

Machines checked (`nvidia-smi -query-gpu=name,driver_version,compute_cap -format=csv`):

- RTX 3050 Laptop GPU — driver 580.173.02, cc 8.6
- RTX A1000 — driver 595.91.07, cc 8.6
- A100 80 GB PCIe — driver 595.91.07, cc 8.0
- T4 (cc 7.5) — not reachable, driver unknown

Toolkit 13.4 (the latest offered) was chosen because every reachable driver is 580 or newer. That a newer 13.x toolkit runs on an older 13.x driver (minor-version compatibility) is from memory; the `sm_86` run on driver 580.173.02 confirms it for that one pairing only. LAMMPS prints **Untested CUDA Toolkit version 13.4**. Use at your own risk at configure time.

```
dnf install -y dnf-plugins-core
dnf config-manager --set-enabled powertools
dnf config-manager --add-repo \
  https://developer.download.nvidia.com/compute/cuda/repos/rhel8/x86_64/cuda-rhel8
  .repo
dnf --disablerepo="*" --enablerepo="cuda*" list 2>/dev/null | \
  grep -E "^cuda-(nvcc|cudart-devel|driver-devel)-(12|13)-"
dnf install -y cuda-nvcc-13-4 cuda-cudart-devel-13-4 cuda-driver-devel-13-4
```

Only `nvcc`, the runtime headers and the driver stub are installed (no `cuda-toolkit-13-4` meta-package, which is large). The listing command confirmed that the package names exist for 12.x

and 13.x. `nvcc` uses `gcc-toolset-14` (which `g++` lists it first); a stray `gcc-c++-8.5.0` was pulled in as a dependency but sits behind it. Persist the environment:

```
cat >> ~/.bashrc <<'EOF'
export PATH=/usr/local/cuda-13.4/bin:/usr/lib64/openmpi/bin:$PATH
export LIBRARY_PATH=/usr/local/cuda-13.4/lib64/stubs:$LIBRARY_PATH
export CUDAToolkit_ROOT=/usr/local/cuda-13.4
EOF
source ~/.bashrc
which nvcc mpicxx
nvcc --version | tail -2
```

Expected: release 13.4, V13.4.92 and `libcuda.so` present in `/usr/local/cuda-13.4/lib64/stubs`.

32.4 uv, Python and shell environment

The image already contained a `pipx`-installed `uv` 0.12.22 that shadowed the fresh install (0.12.23). The duplicate was removed (optional):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv python install 3.12
rm /usr/local/bin/uv /usr/local/bin/uvx
rm -rf /opt/_internal/pipx/venvs/uv
hash -r
which -a uv uvx
```

```
cat >> ~/.bashrc <<'EOF'
export PATH=/root/.local/bin:$PATH
export UVPY=/root/.local/share/uv/python/cpython-3.12-linux-x86_64-gnu/bin/python3
.12
export PYBIN=/opt/python/cp312-cp312/bin
export UV_LINK_MODE=symlink
export UV_CACHE_DIR=/root/.cache/uv
EOF
source ~/.bashrc
$UVPY -c "import sysconfig; print(sysconfig.get_config_var('Py_ENABLE_SHARED'))"
```

The last command must print 1 (shared libpython, Section 4). Both interpreters reported Python 3.12.15. `UV_LINK_MODE=symlink` makes installed packages symlinks into `UV_CACHE_DIR`; deleting that cache breaks them, and symlinks created in `/work` are broken on the host.

33 Configure

```
cd /work && rm -rf build && mkdir build && cd build

cmake -C ../cmake/presets/most.cmake -B . -S ../cmake \
-D BUILD_SHARED_LIBS=yes \
-D CMAKE_BUILD_TYPE=Release \
-D PKG_EXTRA-PAIR=yes \
-D PKG_KIM=yes \
-D PKG_MDI=yes \
-D PKG_PYTHON=yes \
-D PKG_COLVARS=yes \
-D PKG_NETCDF=yes \
-D PKG_VORONOI=yes \
-D PKG_ML-HDNNP=yes \
-D PKG_PLUMED=yes \
-D PKG_ML-QUIP=yes \
-D PKG_ADIOS=no \
-D PKG_QMMM-XTB=no \
-D MLIAP_ENABLE_PYTHON=no \
-D BUILD_MPI=yes \
-D DOWNLOAD_KIM=yes \
-D DOWNLOAD_MDI=yes \
-D DOWNLOAD_N2P2=yes \
-D DOWNLOAD_PLUMED=yes \
-D DOWNLOAD_QUIP=yes \
-D WITH_FFMPEG=yes \
-D Python_EXECUTABLE=$UVPY \
-D PKG_GPU=yes \
-D GPU_API=cuda \
-D GPU_PREC=mixed \
-D GPU_ARCH=sm_86 \
2>&1 | tee ../configure.log | tail -40
```

This stops on FENIX (below). Switch it off in the same build directory, then re-check the options that matter:

```
cmake -D PKG_FENIX=no .
cmake -D PKG_VORONOI=yes .
cmake . > /tmp/cfg.log 2>&1; echo "exit=$?"
grep -E "CMake Error|Configuring done|Generating done" /tmp/cfg.log
grep -i -E "^(PKG_VORONOI|PKG_QMMM-XTB|PKG_FENIX|VORO_LIBRARY|VORO_INCLUDE_DIR|
  CUDPP_OPT)" CMakeCache.txt
```

Expected: exit=0, Configuring done, Generating done, PKG_VORONOI on, PKG_FENIX and PKG_QMMM-XTB off, VORO_LIBRARY=/usr/lib64/libvoro++.so. The GPU block of the summary read: GPU API: CUDA, compiler /usr/local/cuda-13.4/bin/nvcc, default architecture sm_86, CUDPP binning OFF, CUDA MPS OFF, precision MIXED.

33.1 Errors hit, in order

- **GSL:** Could NOT find GSL from Modules/Packages/PLUMED.cmake. Fix: `dnf install -y gsl-devel`.
- **QMMM-XTB:** Checking for module 'xtb>=6.7' failed in pkg-config. Fix: `-D PKG_QMMM-XTB=no`. Why it was enabled at all was not determined; the `most.cmake` preset is the suspect but the preset file was not read.
- **FENIX:** cmake could not find a package configuration file for `fenix` (Modules/Packages/FENIX.cmake). Fix: `cmake -D PKG_FENIX=no .` Same open question about the preset.
- **VORONOI off:** after a clean configure `PKG_VORONOI` was OFF although `-D PKG_VORONOI=yes` was on the command line; cause not determined. `cmake -D PKG_VORONOI=yes .` fixed it. Always check this line before `make`.

Non-blocking: PNetCDF not found; HDF5 “unable to compile a minimal HDF5 program” (HDF5 still found); TBB_MALLOCC not found and a warning that the INTEL package performs best with Intel compilers (the preset enables INTEL); KIM-API not found, so it is downloaded and built; CMake policy CMP0146 and CMP0109 deprecation warnings; the CUDA 13.4 “untested” warning.

33.2 Record: xtb for QMMM-XTB was attempted and dropped

`dnf install xtb` found no package, and `mctc*`, `toml-f*` and `xtb*` are not in the enabled repos. Enabling `PKG_QMMM-XTB` would need `libxtb` 6.7 or newer and `mctc-lib` built from source with Meson (LAMMPS documentation, as read: `-Dinstall_modules=true` so the Fortran module files are installed, plus `pkg-config` files). `xtb` v6.7.1 pins `dftd4` v3.5.0, `s-dftd3` v1.0.0, `multicharge` v0.2.0, `toml-f` v0.4.1, `mstore` v0.2.0 and `test-drive` v0.4.0, and leaves `mctc-lib`, `numsa`, `tblite` and `cpx` at head. A third-party HPC build note found by web search reports GCC 11 and 12 failing on `xtb` 6.7.1 and GCC 13 building with test failures; GCC 14 was not tried. Decision: not needed, dropped. Left behind in the image: `meson` 1.12.1 and `ninja` 1.13.2 (installed with `$PYBIN/python -m pip install` and symlinked into `/usr/local/bin`), and clones of `mctc-lib` and `xtb` v6.7.1 under `/opt/src`. All harmless.

34 Build

The first `make -j12` ended in an error at the link of the `lmp` executable (`liblammps.so` itself had built). Diagnosis, inferred from the link step (the raw error text was not kept): `liblammps.so` records `libcuda.so.1` as a dependency, but the stub directory only provides `libcuda.so`, so the linker cannot resolve it for the executable. `LIBRARY_PATH` only covers `-lcuda` for the library itself. Fix: add a `libcuda.so.1` symlink to the stub directory and pass the directory with `-rpath-link`, which is *not* recorded in the binary, so the stub never ends up in a runtime search path. Apply it before the first `make`:

```
ln -sf libcuda.so /usr/local/cuda-13.4/lib64/stubs/libcuda.so.1
cd /work/build
cmake -D CMAKE_EXE_LINKER_FLAGS="-Wl,-rpath-link,/usr/local/cuda-13.4/lib64/stubs" .
make -j12 2>&1 | tail -25
```

(It was applied after the failed first `make`, which then resumed without recompiling everything. The blunter fallback `-Wl,-allow-shlib-undefined` was not needed.) Expected tail: `Built target lammps, Built target pylammps, Built target lmp`. `-j12` was the value used; no out-of-memory failure occurred.

Pre-wheel check:

```
cd /work/build && ldd liblammps.so | grep -i -E "not found|cuda|nvidia"
```

Result: only `libcuda.so.1 => not found`, which is expected (the real one comes from the NVIDIA driver on the target; it is excluded in Section 35). No `libcudart` entry appeared, so there is nothing CUDA-runtime to vendor (presumably linked statically, not confirmed). Do not run `./lmp` inside the container: it needs the driver's `libcuda.so.1` and will not start.

35 Wheel, auditwheel repair and retag

```
cd /work/python
$PYBIN/python install.py -p lammps -l ../build/liblammps.so \
    -v ../src/version.h -n -w .
$PYBIN/python -m pip install auditwheel patchelf
unzip -l lammps-2026.9.30-cp312-cp312-linux_x86_64.whl | grep -i -E "gnnp|liblammps"
```

Result: `lammps-2026.9.30-cp312-cp312-linux_x86_64.whl`, 48 MB, with `lammps/liblammps.so` (160 MB uncompressed) and `lammps/gnnp_driver.py`.

35.1 First repair: Part A exclude list plus libcuda (fails at runtime)

```
$PYBIN/python -m auditwheel repair \
    lammps-2026.9.30-cp312-cp312-linux_x86_64.whl -w wheelhouse/ \
    --exclude libmpi.so.40 --exclude libmpi_cxx.so.40 \
    --exclude libldap-2.5.so.0 --exclude libcurl.so.4 \
    --exclude libcuda.so.1
```

`libcuda.so.1` must be excluded because it ships with the NVIDIA driver. The repair finished (69.6 MB), but `import lammps` on Pop!_OS aborted in `MPI_Init`, looking for help files under `/usr/lib64/openmpi/share/openmpi/` — the Failure 1 symptom from Part A. `ldd` on the installed `liblammps.so` showed a mixed state: `host libmpi.so.40/libmpi_cxx.so.40` next to vendored copies of `libmpi_mpifh`, `libmpi_usempif08`, `libmpi_usempi_ignore_tkr`, `libopen-rte`, `libopen-pal`, `libopen-orted-mpir`, `libhwloc`, `libevent_core` and `libevent_pthreads` in `lammps.libs/`. The vendored `libopen-pal` is the likely carrier of the AlmaLinux path. Whether those libraries are direct or transitive dependencies of `liblammps.so` was not checked.

35.2 Fix: exclude the whole MPI family

```
cd /work/python
rm -f wheelhouse/lammps-2026.9.30-*.whl
$PYBIN/python -m auditwheel repair \
    lammps-2026.9.30-cp312-cp312-linux_x86_64.whl -w wheelhouse/ \
    --exclude libmpi.so.40 \
    --exclude libmpi_cxx.so.40 \
    --exclude libmpi_mpifh.so.40 \
    --exclude libmpi_usempif08.so.40 \
    --exclude libmpi_usempi_ignore_tkr.so.40 \
    --exclude libopen-rte.so.40 \
    --exclude libopen-pal.so.40 \
    --exclude libopen-orted-mpir.so \
    --exclude libhwloc.so.15 \
    --exclude libevent_core-2.1.so.6 \
    --exclude libevent_pthreads-2.1.so.6 \
    --exclude libldap-2.5.so.0 \
    --exclude libcurl.so.4 \
    --exclude libcuda.so.1
unzip -l wheelhouse/lammps-2026.9.30-*.whl | grep -i -E "mpi|open-|hwloc|event" \
|| echo "no MPI libs bundled"
```

Result: no MPI libs bundled; wheel 68.4MB, tags `manylinux_2_27` and `manylinux_2_28`. `auditwheel` also removed the `libpython3.12.so.1.0` dependency, as in Part A. The host's OpenMPI therefore supplies the whole MPI stack. The `libhwloc` and `libevent` sonames excluded above are the AlmaLinux ones; the host resolves its own versions, which worked on the `Pop!_OS` test machine.

35.3 Retag: drop the Python-version lock

The `lammps` Python module is plain Python that loads `liblammps.so` through `ctypes`, so the `cp312` tag only reflects how `install.py` built the wheel. `wheel tags` writes a second file and keeps the original:

```
cd /work/python/wheelhouse
$PYBIN/python -m pip install -q wheel
$PYBIN/python -m wheel tags --python-tag py3 --abi-tag none \
    lammps-2026.9.30-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
unzip -p lammps-2026.9.30-py3-none-*.whl '*/METADATA' | grep -i -E "(Name|Version|
    Requires-Python)"
unzip -p lammps-2026.9.30-py3-none-*.whl '*/WHEEL' | grep -i "^Tag"
```

Result: `lammps-2026.9.30-py3-none-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl`, `Requires-Python: >=3.6`, tags `py3-none-manylinux_2_27_x86_64` and `py3-none-manylinux_2_28_x86_64`.

Caveat: `PKG_PYTHON` (the `LAMMPS python` command, `fix python/invoke`, ...) was compiled against Python 3.12 headers, and `auditwheel` removed the `libpython` link, so the library uses

whichever interpreter loads it. Across minor versions this may crash; untested, so treat it as safe only under 3.12. Plain ctypes use was tested on 3.12 and 3.10.12 only.

36 Verification

Run on the host (not in the container; the library needs the real `libcuda.so.1`), from `python/wheelhouse`. Python 3.12 with the cp312 wheel, as run:

```
python3.12 -m venv /tmp/lmptest
/tmp/lmptest/bin/pip install -q numpy lammps-2026.9.30-cp312-*.whl
/tmp/lmptest/bin/python -c "
import lammps
l=lammps.lammps(cmdargs=['-log','none','-screen','none'])
f=lambda x: x() if callable(x) else x
print('version:', f(l.version), '| gpu:', f(l.has_gpu_device))
print(l.installed_packages)
"
```

Expected: `version: 20260930 | gpu: True` and the package list. The `f` helper is there because the first script, which called `l.version()` and `l.has_gpu_device()` as methods, died with `TypeError: 'bool' object is not callable: one of the two is a plain attribute in this LAMMPS version (which one was not determined)`. On a machine with a shell `python` shim, call `python3.X` explicitly.

GPU kernel launch test (same venv):

```
/tmp/lmptest/bin/python - 2>&l <<'EOF' | grep -i -E "device|gpu|error|loop time|^
+100 "
import lammps
l = lammps.lammps(cmdargs=['-log','none','-sf','gpu','-pk','gpu','1'])
l.commands_string("""
units lj
lattice fcc 0.8442
region box block 0 10 0 10 0 10
create_box 1 box
create_atoms 1 box
mass 1 1.0
velocity all create 1.44 87287 loop geom
pair_style lj/cut 2.5
pair_coeff 1 1 1.0 1.0 2.5
neighbor 0.3 bin
fix 1 all nve
thermo 50
run 100
""")
EOF
```

`-sf gpu` swaps in `lj/cut/gpu`. Pass criteria: the GPU table lists the device with `Mixed Precision`,

a thermo line for step 100, a `Loop time` line, no `ERROR`. A CUDA error here (unsupported toolchain, no kernel image, invalid PTX) would mean the toolkit does not match the driver. Passed on the RTX 3050 under Python 3.12.

Retagged wheel on a different Python (3.10.12 here), same checks as the first script:

```
python3.10 -m venv /tmp/lmpptest10
/tmp/lmpptest10/bin/pip install -q numpy lammps-2026.9.30-py3-none-*.whl
/tmp/lmpptest10/bin/python -c "
import sys, lammps
l=lammps.lammps(cmdargs=['-log','none','-screen','none'])
f=lambda x: x() if callable(x) else x
print(sys.version.split()[0], '| version:', f(l.version), '| gpu:', f(l.
    has_gpu_device))
"
```

Result: 3.10.12 | version: 20260930 | gpu: True.

37 Target machine requirements for the CUDA wheel

On top of Part C:

- **NVIDIA driver** providing `libcuda.so.1`. Only 580.173.02 was verified with this wheel. The lowest driver that works with a CUDA 13.4 build was not determined; check with `nvidia-smi`.
- **GPU architecture:** `sm_86` verified; `sm_80` and `sm_75` not (Section 28).
- **OpenMPI 4 runtime from the host, including the Fortran bindings and OpenMPI internals** that are now excluded from the wheel (Section 35). Worked on Pop!_OS; which apt package carries the Fortran bindings was not checked.
- `libcurl`, `libldap`, `numpy`, and (for dump movie) `ffmpeg`, as in Part C.

38 Complete command sequence (2026.9.30, CUDA)

Assembled from the commands that were run, with the fixes above applied up front. Not re-run end to end as one script.

```
git fetch --tags -q && git checkout stable_30Sep2026
docker pull quay.io/pypa/manylinux_2_28_x86_64:latest
docker run -it --name lmpbuild -v "$PWD":/work -w /work \
    quay.io/pypa/manylinux_2_28_x86_64:latest

# ---- inside the container ----
cd /work && rm -rf build
dnf install -y openmpi openmpi-devel fftw-devel libjpeg-turbo-devel \
    libpng-devel zlib-devel libcurl-devel netcdf-devel openblas-devel \
    lapack-devel ncurses-devel git
```

```

dnf install -y epel-release
dnf install -y https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.
noarch.rpm
dnf install -y ffmpeg
dnf install -y voro++ voro++-devel eigen3-devel gsl-devel libzstd-devel

curl -fL -o /tmp/cmake.tgz https://github.com/Kitware/CMake/releases/download/v4
.4.3/cmake-4.4.3-linux-x86_64.tar.gz
mkdir -p /opt/cmake-4.4.3 && tar -xzf /tmp/cmake.tgz -C /opt/cmake-4.4.3 --strip-
components=1
for t in cmake ccmake ctest cpack; do ln -sf /opt/cmake-4.4.3/bin/$t /usr/local/bin/
$t; done

dnf install -y dnf-plugins-core
dnf config-manager --set-enabled powertools
dnf config-manager --add-repo https://developer.download.nvidia.com/compute/cuda/
repos/rhel8/x86_64/cuda-rhel8.repo
dnf install -y cuda-nvcc-13-4 cuda-cudart-devel-13-4 cuda-driver-devel-13-4
ln -sf libcuda.so /usr/local/cuda-13.4/lib64/stubs/libcuda.so.1

curl -LsSf https://astral.sh/uv/install.sh | sh && source $HOME/.local/bin/env
uv python install 3.12
cat >> ~/.bashrc <<'EOF'
export PATH=/usr/local/cuda-13.4/bin:/usr/lib64/openmpi/bin:/root/.local/bin:$PATH
export LIBRARY_PATH=/usr/local/cuda-13.4/lib64/stubs:$LIBRARY_PATH
export CUDAToolkit_ROOT=/usr/local/cuda-13.4
export UVPY=/root/.local/share/uv/python/cpython-3.12-linux-x86_64-gnu/bin/python3
.12
export PYBIN=/opt/python/cp312-cp312/bin
export UV_LINK_MODE=symlink
export UV_CACHE_DIR=/root/.cache/uv
EOF
source ~/.bashrc

mkdir build && cd build
cmake -C ../cmake/presets/most.cmake -B . -S ../cmake \
-D BUILD_SHARED_LIBS=yes -D CMAKE_BUILD_TYPE=Release \
-D PKG_EXTRA-PAIR=yes -D PKG_KIM=yes -D PKG_MDI=yes -D PKG_PYTHON=yes \
-D PKG_COLVARS=yes -D PKG_NETCDF=yes -D PKG_VORONOI=yes \
-D PKG_ML-HDNNP=yes -D PKG_PLUMED=yes -D PKG_ML-QUIP=yes \
-D PKG_ADIOS=no -D PKG_QMMM-XTB=no -D MLIAP_ENABLE_PYTHON=no \
-D BUILD_MPI=yes -D DOWNLOAD_KIM=yes -D DOWNLOAD_MDI=yes \
-D DOWNLOAD_N2P2=yes -D DOWNLOAD_PLUMED=yes -D DOWNLOAD_QUIP=yes \
-D WITH_FFMPEG=yes -D Python_EXECUTABLE=$UVPY \
-D PKG_GPU=yes -D GPU_API=cuda -D GPU_PREC=mixed -D GPU_ARCH=sm_86
cmake -D PKG_FENIX=no .
cmake -D PKG_VORONOI=yes .
grep -E "(PKG_VORONOI|PKG_FENIX|PKG_QMMM-XTB)" CMakeCache.txt

cmake -D CMAKE_EXE_LINKER_FLAGS="-Wl,-rpath-link,/usr/local/cuda-13.4/lib64/stubs" .
make -j12
ldd liblammps.so | grep -i -E "not found|cuda|nvidia"

```

```

cd /work/python
$PYBIN/python install.py -p lammps -l ../build/liblammps.so -v ../src/version.h -n -
w .
$PYBIN/python -m pip install auditwheel patchelf wheel
mkdir -p wheelhouse
$PYBIN/python -m auditwheel repair lammps-2026.9.30-cp312-cp312-linux_x86_64.whl \
-w wheelhouse/ \
--exclude libmpi.so.40 --exclude libmpi_cxx.so.40 \
--exclude libmpi_mpifh.so.40 --exclude libmpi_usempif08.so.40 \
--exclude libmpi_usempi_ignore_tkr.so.40 --exclude libopen-rte.so.40 \
--exclude libopen-pal.so.40 --exclude libopen-orted-mpir.so \
--exclude libhwloc.so.15 --exclude libevent_core-2.1.so.6 \
--exclude libevent_pthreads-2.1.so.6 --exclude libldap-2.5.so.0 \
--exclude libcurl.so.4 --exclude libcuda.so.1
cd wheelhouse
$PYBIN/python -m wheel tags --python-tag py3 --abi-tag none \
lammps-2026.9.30-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl

```

A Unrelated: A100 CUDA export fix

From a separate GPU/MLIP debugging session on an A100 machine (SevenNet’s D3 dispersion-correction JIT-compiles a CUDA extension at runtime). Not part of the CPU wheel build.

Symptom: `nvcc fatal: Unsupported gpu architecture 'compute_61'` — CUDA 13.0 `nvcc` refuses old Pascal-era `sm_61` in the default wide arch list, even though the actual GPU (A100 = `sm_80`) is fine.

Fix, on the A100 box, before running:

```
export TORCH_CUDA_ARCH_LIST="8.0"
```

Scopes the build to just the A100’s compute capability, so `nvcc` never touches the unsupported old arch. Session-only — add to `venv activate script` or `.bashrc` if persistence is needed.